Software firm specializing in high-performance Rust-based applications, Applied Cryptography, Applied Machine Learning (Fine-tuning, quantization, RAG), and secure communications.

Follow publication

DGX Spark, Nemotron3, and NVFP4: Getting to 65+ tps

Thomas P. Braun Follow 4 min read · Dec 20, 2025

13 1

DGX Spark, Nemotron3, and NVFP4:
Getting to 65+ tps
42 tps
67+ tps
CUDA graphs
kernel compatibility
container crashes
Accelerated

Running state-of-the-art language models on NVIDIA's DGX Spark isn't as straightforward as `docker run`. After spending considerable time wrestling with CUDA graphs, kernel compatibility issues, and mysterious container crashes, I've documented the complete journey from 42 tokens/second to 67+ tokens/second with full CUDA graph acceleration. Skip to the end for a copy and paste command to have the fastest quantization of the nemotron3 model running.

The Challenge

The DGX Spark features NVIDIA's GB10 "Grace Blackwell" chip — a cutting-edge SoC combining an ARM-based Grace CPU with a Blackwell GPU at compute capability 12.1. This is bleeding-edge hardware, and the software ecosystem is still catching up.

I wanted to run **Nemotron-3-Nano** (specifically the NVFP4 quantized version by cybermotaz) — a hybrid Mamba-Transformer model that promises impressive performance. The model uses:

- **NemotronH architecture**: A hybrid SSM (State Space Model) combining Mamba blocks with Transformer attention
- **NVFP4 quantization:** 4-bit floating point quantization via NVIDIA's modelopt
- **Non-gated activations**: ReLU-squared instead of traditional SiLU/GELU

Attempt #1: Stock vLLM Docker Image

```
docker run --gpus all vllm/vllm-openai:latest \
  serve cybermotaz/nemotron3-nano-nvfp4-w4a16 \
  --quantization modelopt_fp4
```

Result: Complete failure. The stock vLLM image is built for older CUDA architectures and doesn't support SM 12.1 (Blackwell).

Attempt #2: Building vLLM from Source

I built vLLM from the main branch with `TORCH_CUDA_ARCH_LIST="12.1f"`. The `f` suffix enables forward compatibility via PTX JIT compilation.

Get Thomas P. Braun's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

☑ Remember me for faster sign in

New Error:

```
ValueError: NemotronHForCausalLM uses non-gated activation,
which is not supported by FusedMoE
```

The culprit? Nemotron-3-Nano uses **ReLU-squared** activation instead of the gated activations (SiLU) that vLLM's fused MoE kernels expected.

The Non-Gated Activation Fix

Fortunately, PR #29004 added support for non-gated activations. Building from the main branch (post-merge) resolved this issue.

Attempt #3: CUDA Graphs and the SM120 CUTLASS Problem

With non-gated activations working, I tried enabling CUDA graphs for maximum performance:

```
docker run --gpus all my-vllm-image \
  serve cybermotaz/nemotron3-nano-nvfp4-w4a16 \
  --quantization modelopt_fp4 \
  --max-model-len 131072 \
  --gpu-memory-utilization 0.85
```

Result: Container crash during CUDA graph capture with this cryptic error:

```
RuntimeError: Failed to initialize cutlass TMA WS grouped gemm
```

The FlashInfer MoE backend was trying to use CUTLASS kernels that don't fully support SM 12.1 in throughput mode.

The Magic Fix: FlashInfer Backend Mode

After extensive debugging, I discovered the solution:

```
export VLLM_FLASHINFER_MOE_BACKEND=latency
```

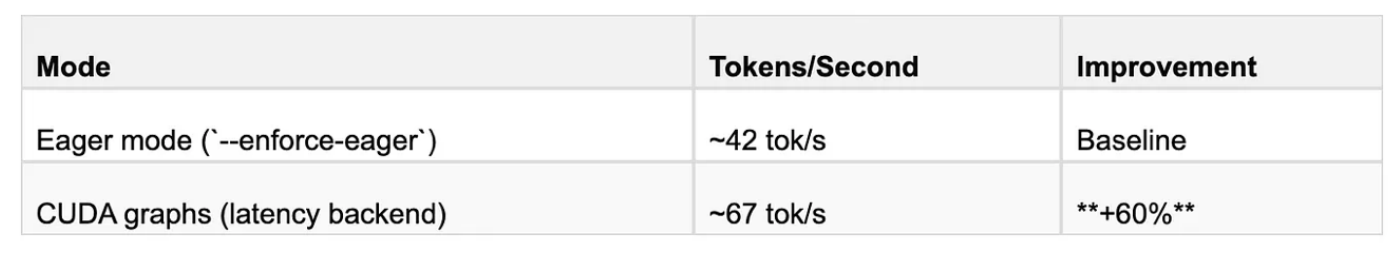
Instead of `throughput` mode (which uses problematic CUTLASS grouped GEMM kernels), `latency` mode uses a different kernel path that's compatible with SM 12.1.

The Final Working Configuration

Here's the one-liner that achieves 65+ tokens/second with full CUDA graph acceleration, including reasoning and tool calling support:

```
docker run --rm -it --gpus all --ipc=host -p 8000:8000 \
  -e VLLM_FLASHINFER_MOE_BACKEND=latency \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  avarok/vllm-dgx-spark:v11 \
  serve cybermotaz/nemotron3-nano-nvfp4-w4a16 \
  --quantization modelopt_fp4 \
  --kv-cache-dtype fp8 \
  --trust-remote-code \
  --max-model-len 131072 \
  --gpu-memory-utilization 0.85 \
  --enable-auto-tool-choice \
  --tool-call-parser qwen3_coder \
  --reasoning-parser deepseek_r1
```

Performance Results



Mode	Tokens/Second	Improvement
Eager mode (`--enforce-eager`)	~42 tok/s	Baseline
CUDA graphs (latency backend)	~67 tok/s	**+60%**

That's a 60% performance improvement just from enabling CUDA graphs with the correct backend configuration.

Building Your Own Image

The Docker image is available at `avarok/vllm-dgx-spark:v11`, but if you want to build it yourself, here's the key Dockerfile configuration:

```
FROM nvidia/cuda:13.0.2-cudnn-devel-ubuntu24.04
```

```
# Critical: Set architecture for Blackwell GPU
ENV TORCH_CUDA_ARCH_LIST="12.1f"

# Critical: Use latency backend for SM12.1 compatibility
ENV VLLM_FLASHINFER_MOE_BACKEND=latency

# Install PyTorch with CUDA 13.0 support
RUN pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu130

# Install FlashInfer for CUDA 13.0
RUN pip install flashinfer-python -i
https://flashinfer.ai/whl/cu130/torch2.7

# Build vLLM from main branch
RUN git clone https://github.com/vllm-project/vllm.git && \
    cd vllm && \
    pip install --no-build-isolation . -v
```

The full Dockerfile and build instructions are available on HuggingFace at Avarok/vllm-dgx-spark.

Key Configuration Parameters Explained

- **`--quantization modelopt_fp4`**: Uses NVIDIA's FP4 quantization format
- **`--kv-cache-dtype fp8`**: FP8 KV cache reduces memory usage while maintaining quality
- **`--max-model-len 131072`**: 128K context window (ideal for the DGX Spark)
- **`--gpu-memory-utilization 0.85`**: Leave headroom for CUDA graph capture
- **`--ipc=host`**: Required for proper shared memory access
- **`VLLM_FLASHINFER_MOE_BACKEND=latency`**: The critical fix for SM12.1 compatibility
- **`--enable-auto-tool-choice`**: Enables automatic function/tool calling for agentic workflows
- **`--tool-call-parser qwen3_coder`**: Parser for structured tool call responses
- **`--reasoning-parser deepseek_r1`**: Parser for reasoning tokens (think tags)

Troubleshooting Common Issues

"Free memory increased during profiling"

Reduce `--gpu-memory-utilization` to 0.75-0.80. This happens when torch.compile releases buffers during memory profiling.

Container crashes during CUDA graph capture

Ensure `VLLM_FLASHINFER_MOE_BACKEND=latency` is set. The throughput backend uses CUTLASS kernels that don't work on SM12.1.

Model loading fails with "non-gated activation" error

You need vLLM from the main branch (after May 2025) which includes PR #29004.

Conclusion

Getting optimal performance on DGX Spark requires understanding the nuances of CUDA 13.0, Blackwell architecture, and vLLM's various backend options. The key discoveries were:

1. **Build for SM 12.1f** with forward compatibility
2. **Use the latency backend** for FlashInfer MoE operations
3. **Leave GPU memory headroom** for CUDA graph capture
4. **Build from vLLM main** for non-gated activation support

With these configurations, the DGX Spark delivers impressive 67 tokens/second inference on Nemotron-3-Nano — making it a capable edge AI platform for production workloads.

. . .

Credits to cybermotaz for the excellent NVFP4 quantization of Nemotron-3-Nano.

Docker image: `avarok/vllm-dgx-spark:v11` *HuggingFace repo: Avarok/vllm-dgx-spark*

AI Nvidia Dgx Spark Nemotron Software Engineering

13 1

Published in Avarok
21 followers · Last published Feb 20, 2026
Software firm specializing in high-performance Rust-based applications, Applied Cryptography, Applied Machine Learning (Fine-tuning, quantization, RAG), and secure communications.
Follow

Written by Thomas P. Braun
50 followers · 13 following
Philosopher | Inventor | Applied Cryptographer | Founder @ Avarok
Follow

Responses (1)

Write a response

What are your thoughts?

Cédric M.
Jan 5

Thank you for the Docker image; it works perfectly, and I can confirm the 60% performance improvement with Nemotron 3. I've just started experimenting with my Spark, and resources like this one are incredibly helpful—I'd even call it a gem.

I noticed… more

2 replies Reply

More from Thomas P. Braun and Avarok

In Avarok by Thomas P. Braun
We Unlocked NVFP4 on DGX Spark — and It's 20% Faster Than AWQ
NVIDIA's own software stack couldn't do it. We did. Avarok's open-source vLLM image i…
Feb 20 35 4

In Avarok by Thomas P. Braun
NVFP4 W4A4 MoE Inference on NVIDIA Blackwell GB10
A Deep Dive on Nvidia DGX
Dec 10, 2025 7 1

In Avarok by Thomas P. Braun
LatticeDB: The Database That Makes AI Apps Actually Work
A hybrid vector-graph database that runs everywhere—from your browser to the edge
Jan 26

In Avarok by Thomas P. Braun
From 20 to 35 Tokens/Second: Optimizing NVFP4 Inference on…
Introduction: Getting 35 tps from Qwen3-Next on an Nvidia DGX Spark
Dec 30, 2025 1 1

See all from Thomas P. Braun See all from Avarok

Recommended from Medium

Doran Gao
Connecting Two DGX Spark Systems via 200Gb/s RoCE…
Growth is never by mere chance; it is the result of forces working together.—James…
Oct 30, 2025 77 3

In Towards Dev by Bill WANG
Running OpenClaw with a Local LLM on a Mac mini (No API Cost)
Feb 15 144 2

In CodeX by MayhemCode
Why Thousands Are Buying Mac Minis to Escape Issues with Big…
Something strange happened in early 2026. Apple stores started running low on Mac…
Feb 15 5K 83

In Level Up Coding by Teja Kusireddy
Google Is Quietly Dismantling Everything OpenAI Built
The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine…
Feb 17 5.5K 166

Agent Native
OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cogne…
If your agent has ever randomly ignored a decision you know you told it… it's not…
Feb 20 261 5

In Sparktastic by Amegilla
Starting with a Spark
My Adventures Building a Home AI Lab with a Desktop AI Supercomputer
Nov 14, 2025 12

See more recommendations